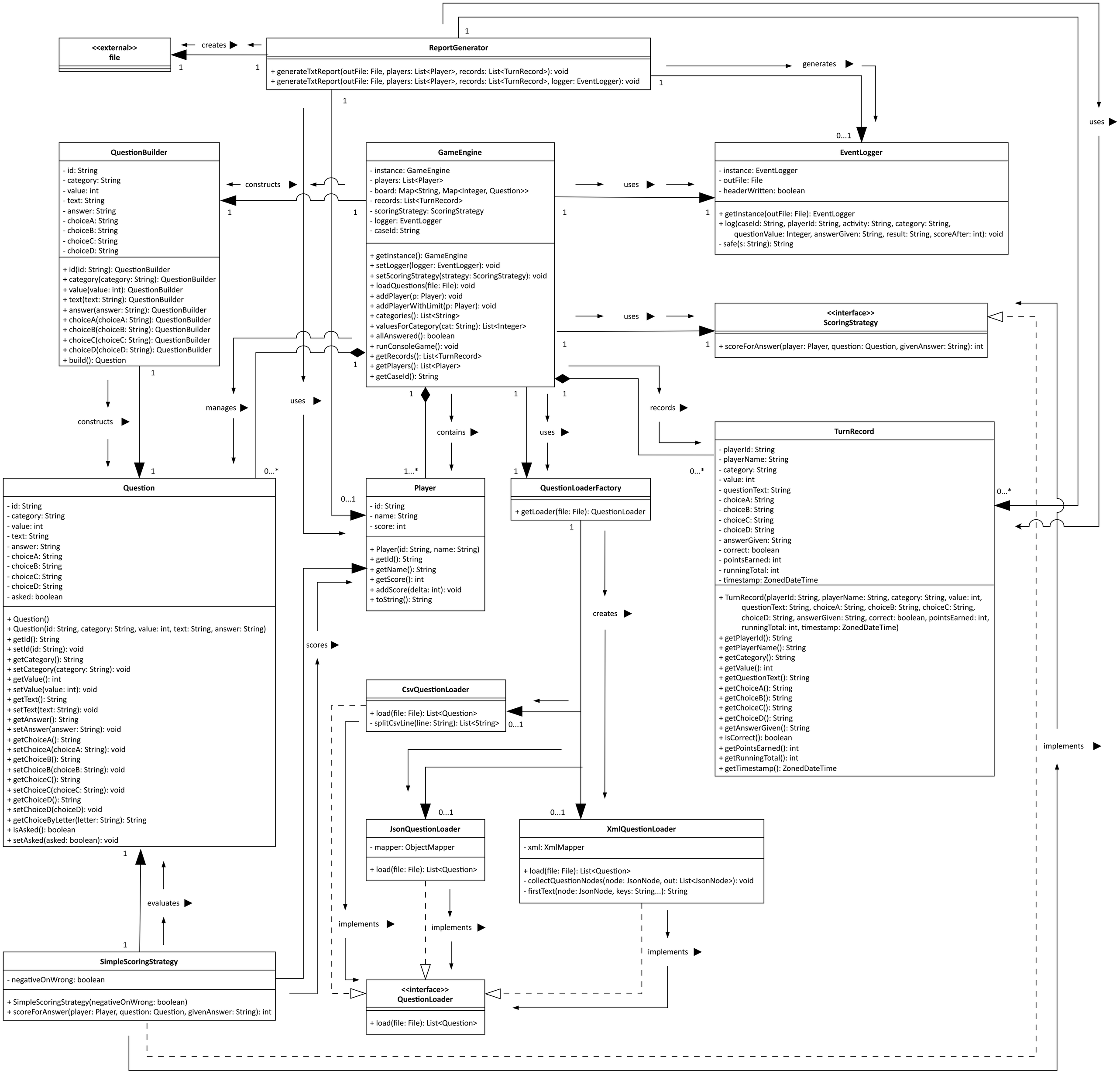
<<external>>
file
creates
ReportGenerator
+ generateTxtReport(outFile: File, players: List<Player>, records: List<TurnRecord>): void
+ generateTxtReport(outFile: File, players: List<Player>, records: List<TurnRecord>, logger: EventLogger): void
generates
uses
QuestionBuilder
- id: String
- category: String
- value: int
- text: String
- answer: String
- choiceA: String
- choiceB: String
- choiceC: String
- choiceD: String
+ id(id: String): QuestionBuilder
+ category(category: String): QuestionBuilder
+ value(value: int): QuestionBuilder
+ text(text: String): QuestionBuilder
+ answer(answer: String): QuestionBuilder
+ choiceA(choiceA: String): QuestionBuilder
+ choiceB(choiceB: String): QuestionBuilder
+ choiceC(choiceC: String): QuestionBuilder
+ choiceD(choiceD: String): QuestionBuilder
+ build(): Question
constructs
GameEngine
- instance: GameEngine
- players: List<Player>
- board: Map<String, Map<Integer, Question>>
- records: List<TurnRecord>
- scoringStrategy: ScoringStrategy
- logger: EventLogger
- caseId: String
+ getInstance(): GameEngine
+ setLogger(logger: EventLogger): void
+ setScoringStrategy(strategy: ScoringStrategy): void
+ loadQuestions(file: File): void
+ addPlayer(p: Player): void
+ addPlayerWithLimit(p: Player): void
+ categories(): List<String>
+ valuesForCategory(cat: String): List<Integer>
+ allAnswered(): boolean
+ runConsoleGame(): void
+ getRecords(): List<TurnRecord>
+ getPlayers(): List<Player>
+ getCaseId(): String
EventLogger
- instance: EventLogger
- outFile: File
- headerWritten: boolean
+ getInstance(outFile: File): EventLogger
+ log(caseId: String, playerId: String, activity: String, category: String,
questionValue: Integer, answerGiven: String, result: String, scoreAfter: int): void
- safe(s: String): String
<<interface>>
ScoringStrategy
+ scoreForAnswer(player: Player, question: Question, givenAnswer: String): int
manages
records
contains
TurnRecord
- playerId: String
- playerName: String
- category: String
- value: int
- questionText: String
- choiceA: String
- choiceB: String
- choiceC: String
- choiceD: String
- answerGiven: String
- correct: boolean
- pointsEarned: int
- runningTotal: int
- timestamp: ZonedDateTime
+ TurnRecord(playerId: String, playerName: String, category: String, value: int,
questionText: String, choiceA: String, choiceB: String, choiceC: String,
choiceD: String, answerGiven: String, correct: boolean, pointsEarned: int,
runningTotal: int, timestamp: ZonedDateTime)
+ getPlayerId(): String
+ getPlayerName(): String
+ getCategory(): String
+ getValue(): int
+ getQuestionText(): String
+ getChoiceA(): String
+ getChoiceB(): String
+ getChoiceC(): String
+ getChoiceD(): String
+ getAnswerGiven(): String
+ isCorrect(): boolean
+ getPointsEarned(): int
+ getRunningTotal(): int
+ getTimestamp(): ZonedDateTime
Question
- id: String
- category: String
- value: int
- text: String
- answer: String
- choiceA: String
- choiceB: String
- choiceC: String
- choiceD: String
- asked: boolean
+ Question()
+ Question(id: String, category: String, value: int, text: String, answer: String)
+ getId(): String
+ setId(id: String): void
+ getCategory(): String
+ setCategory(category: String): void
+ getValue(): int
+ setValue(value: int): void
+ getText(): String
+ setText(text: String): void
+ getAnswer(): String
+ setAnswer(answer: String): void
+ getChoiceA(): String
+ setChoiceA(choiceA: String): void
+ getChoiceB(): String
+ setChoiceB(choiceB: String): void
+ getChoiceC(): String
+ setChoiceC(choiceC: String): void
+ getChoiceD(): String
+ setChoiceD(choiceD): void
+ getChoiceByLetter(letter: String): String
+ isAsked(): boolean
+ setAsked(asked: boolean): void
Player
- id: String
- name: String
- score: int
+ Player(id: String, name: String)
+ getId(): String
+ getName(): String
+ getScore(): int
+ addScore(delta: int): void
+ toString(): String
QuestionLoaderFactory
+ getLoader(file: File): QuestionLoader
scores
creates
CsvQuestionLoader
+ load(file: File): List<Question>
- splitCsvLine(line: String): List<String>
JsonQuestionLoader
- mapper: ObjectMapper
+ load(file: File): List<Question>
XmlQuestionLoader
- xml: XmlMapper
+ load(file: File): List<Question>
- collectQuestionNodes(node: JsonNode, out: List<JsonNode>): void
- firstText(node: JsonNode, keys: String...): String
implements
evaluates
SimpleScoringStrategy
- negativeOnWrong: boolean
+ SimpleScoringStrategy(negativeOnWrong: boolean)
+ scoreForAnswer(player: Player, question: Question, givenAnswer: String): int
<<interface>>
QuestionLoader
+ load(file: File): List<Question>
1
0...1
0...*
1...*